

Week Summary

Factors affecting parallel performance

Performance degradation

SAB: section 1.4.5

- A number of factors can contribute to real-world parallel performance being less than the idealised peak performance:
 - **S**tarvation: insufficient parallel work to keep the processors busy (insufficient parallel work, uneven distribution of work)
 - **L**atency: time taken for information to travel from one part of the system to another (e.g. accessing memory, sending messages)
 - **O**verhead: work required in addition to the computation (e.g. starting and stopping OpenMP parallel regions)
 - **W**aiting: multiple threads/processes accessing a shared resource (e.g. contention for memory or network bandwidth)

Starvation

- We saw an example of starvation in workshop 2 (insufficient loop iterations)
- This week we looked at the impact of parallel fraction (Amdahl's law and Gustafson's law)
- Load balancing can also cause performance reduction due to starvation
- The workshop this week looks at load balancing and loop scheduling

```
/* Calculate values for x and y by looping over the parameter t */
#pragma omp parallel for default(shared) private(t)
for (int curve=0; curve<N_CURVES; curve++){
    for (int i=0; i<N_POINTS; i++){
        printf("Thread %d: curve=%d, i=%d\n", omp_get_thread_num(), curve, i);
        /* Convert i and N to floats so we do floating point division */
        t = 2.0*PI * (float) i / (float) N_POINTS;
        x[curve][i] = A[curve] * sin( omega_x[curve]*t + delta_x[curve] );
        y[curve][i] = B[curve] * sin( omega_y[curve]*t + delta_y[curve] );
    }
}
```

N_CURVES=4
OMP_NUM_THREADS=8

Latency

- Last week we saw how the latency accessing main memory reduces performance below the theoretical peak for a memory bound algorithm
- In unit 7.3 we looked at the latency associated with sending an MPI message
- In unit 7.4 we looked at message size required for halo exchange in a domain decomposition and how this scales with sub-domain size

Overhead

- There are a number of overheads associated with running a program in parallel
- For example in MPI programs we need to divide up the work between the processes
- In OpenMP there are overheads associated with starting and stopping parallel regions and loop scheduling
- Dynamic loop scheduling allows load balancing at run time but has a higher overhead than static scheduling
- In unit 7.5 we saw how to incorporate an overhead term into Amdahl's law

Waiting

- Waiting a performance reduction when multiple threads or processes are accessing a shared resource
- Examples are contention for memory or network bandwidth
- If performance is limited by waiting then adding more processor cores will not result in a speed-up
- Processor cores add compute resources but do not increase the resource which is the limiting factor (e.g. memory bandwidth)